

Diving into Redux

Easy Guide

Contents

1	Introducing Redux Toolkit	2
1.1	What is Redux Toolkit?	2
1.1.1	Before and After	2
1.2	Key Benefits	2
1.3	What Redux Toolkit Includes	3
2	Redux vs React Context	4
2.1	Two Ways to Manage State	4
2.2	React Context - Simple Approach	4
2.3	Redux - Powerful Approach	5
2.4	Comparison Table	5
3	How Redux Works	6
3.1	The Redux Cycle	6
3.2	Redux Parts	6
3.2.1	1. State	6
3.2.2	2. Actions	6
3.2.3	3. Reducers	7
3.2.4	4. Store	7
3.3	Visual Flow	7
4	MUST READ: Redux createStore() is deprecated?	8
4.1	The Old Way: createStore()	8
4.2	Why It's Deprecated	8
4.3	The New Way: configureStore()	8
4.4	Why configureStore() is Better	9
4.5	What Changed	9
4.6	If You See createStore()	9
5	Exploring The Core Redux Concepts	10
5.1	Five Core Concepts	10
5.1.1	1. Store	10
5.1.2	2. State	10
5.1.3	3. Actions	10
5.1.4	4. Reducers	11
5.1.5	5. Dispatch	11
5.2	How They Connect	11

6	Redux Basics	12
6.1	The Three Rules of Redux	12
6.1.1	Rule 1: Single State Object	12
6.1.2	Rule 2: State is Read-Only	12
6.1.3	Rule 3: Changes via Pure Functions	12
6.2	Pure Functions	13
6.2.1	Good Examples	13
6.2.2	Bad Examples	13
7	Creating a Redux Store for React	14
7.1	Step-by-Step Setup	14
7.1.1	Step 1: Create a Slice	14
7.1.2	Step 2: Create the Store	14
7.2	File Structure	15
8	Providing the Store	16
8.1	Making Store Available	16
8.2	In main.jsx or index.js	16
8.3	Why Provider?	16
9	Using Redux Data in React Components	18
9.1	Reading State from Redux	18
9.2	Understanding useSelector	18
9.3	Different Selectors	18
9.4	Multiple Values	19
10	Dispatching Actions From Inside Components	20
10.1	Sending Actions	20
10.2	How It Works	20
10.3	Complete Example	20
10.4	Multiple Dispatches	21
11	Redux with Class-based Components	22
11.1	Connecting Class Components	22
11.2	mapStateToProps	23
11.3	mapDispatchToProps	23
12	Attaching Payloads to Actions	24
12.1	What is a Payload?	24
12.2	Creating Actions with Payloads	24
12.3	Using Payloads in Components	25
12.4	Complex Payloads	25
13	Working with Multiple State Properties	26
13.1	Single Slice, Multiple Properties	26
13.2	Selecting Multiple Properties	26
13.3	Select Entire Object	27
13.4	Multiple Slices	27
13.5	Accessing Multiple Slices	27

14 How To Work With Redux State Correctly	28
14.1 The Immutability Rule	28
14.2 Redux Toolkit Simplifies This	28
14.3 Common Patterns	29
14.3.1 Adding to Array	29
14.3.2 Removing from Array	29
14.3.3 Updating Nested Objects	29
15 Adding State Slices	31
15.1 What is a Slice?	31
15.2 Creating a New Slice	31
15.3 Adding to Store	32
15.4 Using the Slice	32
16 Connecting Redux Toolkit State	34
16.1 Step 1: Create Slice	34
16.2 Step 2: Add to Store	34
16.3 Step 3: Use in Component	35
17 Working with Multiple Slices	37
17.1 Many Slices Strategy	37
17.2 Each Slice Example	37
17.3 Combining in Store	38
17.4 State Shape	38
17.5 Using Multiple Slices	38
18 Reading & Dispatching From A New Slice	39
18.1 Complete Example: Todo Slice	39
18.1.1 Create Todo Slice	39
18.1.2 Add to Store	40
18.1.3 Use in Component	40

Chapter 1

Introducing Redux Toolkit

1.1 What is Redux Toolkit?

Redux Toolkit is a modern, easier way to use Redux. It simplifies Redux and reduces the amount of code you need to write.

1.1.1 Before and After

Without Redux Toolkit (Old Way):

- Write lots of boilerplate code
- Create actions manually
- Create reducers manually
- Set up the store manually
- Many files and configurations

With Redux Toolkit (New Way):

- Write much less code
- Everything is automated
- Simple configuration
- Faster development
- Less mistakes

1.2 Key Benefits

1. **Simpler** - Less code to write
2. **Faster** - Get started quickly
3. **Built-in tools** - Includes common utilities
4. **Better performance** - Optimized automatically
5. **Recommended** - Official Redux recommendation

1.3 What Redux Toolkit Includes

- `configureStore()` - Set up the store
- `createSlice()` - Create actions and reducers
- `createAsyncThunk()` - Handle async operations
- Immer integration - Update state immutably
- Redux DevTools - Debug your app

Recommendation: Always use Redux Toolkit for new projects!

Chapter 2

Redux vs React Context

2.1 Two Ways to Manage State

React gives you two main approaches for sharing state:

1. React Context API
2. Redux

2.2 React Context - Simple Approach

Best for:

- Small to medium apps
- Simple state (theme, user)
- Quick projects
- Learning React

Pros:

- Built into React
- Easy to learn
- Less setup
- No extra library

Cons:

- Can be slow with lots of updates
- Hard to debug
- State updates all consumers
- Not good for complex state

2.3 Redux - Powerful Approach

Best for:

- Large applications
- Complex state
- Many state changes
- Team projects

Pros:

- Very performant
- Easy to debug
- Time-travel debugging
- Predictable state changes
- Good for teams

Cons:

- More setup needed
- More to learn
- More files to create
- Can be overkill for small apps

2.4 Comparison Table

Feature	Context	Redux
Setup difficulty	Easy	Medium
Code amount	Less	More
Performance	Slower	Faster
Debugging	Hard	Easy
Best use	Small apps	Large apps
Learning curve	Low	Medium

Choice: Use Context for simple state, Redux for complex apps.

Chapter 3

How Redux Works

3.1 The Redux Cycle

Redux follows a clear, predictable cycle:

1. User interacts with app
2. Component dispatches an action
3. Action goes to the reducer
4. Reducer updates the state
5. Store notifies all subscribed components
6. Components re-render with new state

3.2 Redux Parts

3.2.1 1. State

The central data storage. It holds all your app's data.

```
{  
  counter: 0,  
  user: { name: 'John', age: 25 },  
  items: [...]  
}
```

3.2.2 2. Actions

Objects that describe what happened.

```
{ type: 'INCREMENT' }  
{ type: 'SET_USER', payload: { name: 'Jane' } }
```

3.2.3 3. Reducers

Functions that update state based on actions.

```
function counterReducer(state = 0, action) {  
  if (action.type === 'INCREMENT') {  
    return state + 1;  
  }  
  return state;  
}
```

3.2.4 4. Store

Container that holds state, actions, and reducers together.

```
const store = createStore(reducer);
```

3.3 Visual Flow

```
Component  
  ↓ (Dispatch Action)  
Action: { type: 'ADD_ITEM' }  
  ↓  
Reducer  
  ↓ (Updates State)  
Store  
  ↓ (Notifies subscribers)  
Component (Re-renders with new state)
```

Key Concept: Actions describe what happened, reducers update state!

Chapter 4

MUST READ: Redux createStore() is deprecated?

4.1 The Old Way: createStore()

createStore() was the original way to create Redux stores:

```
import { createStore } from 'redux';

const store = createStore(rootReducer);
```

4.2 Why It's Deprecated

Redux removed createStore() because:

1. Too much boilerplate code
2. Hard to set up correctly
3. Missing helpful features
4. Modern tools are better

4.3 The New Way: configureStore()

Redux Toolkit's configureStore() is the modern replacement:

```
import { configureStore } from '@reduxjs/toolkit';

const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer
  }
});
```

4.4 Why `configureStore()` is Better

- Less code
- Better defaults
- Built-in Redux DevTools
- Automatic middleware setup
- Handles async operations

4.5 What Changed

Old (<code>createStore</code>)	New (<code>configureStore</code>)
Manual setup	Automatic setup
Missing features	Includes everything
No DevTools	Built-in DevTools
Hard to debug	Easy to debug

Important: Don't use `createStore()` in new projects. Use `configureStore()` instead!

4.6 If You See `createStore()`

If you're reading old code or docs:

- It still works but is outdated
- Migrate to `configureStore()`
- Modern tutorials use Redux Toolkit

Chapter 5

Exploring The Core Redux Concepts

5.1 Five Core Concepts

5.1.1 1. Store

The big container holding everything.

```
const store = configureStore({
  reducer: rootReducer
});
```

What it does:

- Holds the state
- Has methods like `dispatch()` and `subscribe()`
- Notifies listeners when state changes

5.1.2 2. State

All the data your app needs.

```
state = {
  counter: 5,
  user: { id: 1, name: 'John' }
}
```

5.1.3 3. Actions

Objects describing what happened:

```
const incrementAction = { type: 'INCREMENT' };
const addUserAction = {
  type: 'ADD_USER',
  payload: { name: 'Jane', age: 30 }
};
```

5.1.4 4. Reducers

Pure functions that update state:

```
(state, action) => newState
```

5.1.5 5. Dispatch

Sending an action to the store:

```
store.dispatch({ type: 'INCREMENT' });
```

5.2 How They Connect

`store.dispatch(action) → reducer → state updates`

Remember: State is changed **ONLY** through reducers!

Chapter 6

Redux Basics

6.1 The Three Rules of Redux

6.1.1 Rule 1: Single State Object

Keep all state in one place. Don't scatter it everywhere.

```
// Good: One store
const store = {
  counter: 0,
  user: { name: 'John' },
  items: [...]
}
```

```
// Bad: State everywhere
let counter = 0;
let user = {};
let items = [];
```

6.1.2 Rule 2: State is Read-Only

Never modify state directly. Only through actions and reducers.

```
// Bad: Direct modification
state.counter = state.counter + 1;

// Good: Via reducer
reducer returns { ...state, counter: state.counter + 1 }
```

6.1.3 Rule 3: Changes via Pure Functions

Reducers must be pure functions. Same input = same output.

```
// Bad: Not pure (depends on external variable)
let bonus = 5;
function add(a) { return a + bonus; }
```

```
// Good: Pure
function add(a) { return a + 5; }
```

6.2 Pure Functions

A pure function:

- Returns the same output for same input
- Doesn't modify external variables
- Doesn't make API calls
- Doesn't produce side effects

6.2.1 Good Examples

```
// Pure: Predictable
function double(x) { return x * 2; }
function add(a, b) { return a + b; }
```

6.2.2 Bad Examples

```
// Not pure: Unpredictable
let count = 0;
function increment() { return ++count; } // Changes external

function randomNumber() { return Math.random(); } // Different each time
```

Principle: Predictability is Redux's superpower!

Chapter 7

Creating a Redux Store for React

7.1 Step-by-Step Setup

7.1.1 Step 1: Create a Slice

A slice combines actions and reducer:

```
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => {
      state.value += 1;
    },
    decrement: (state) => {
      state.value -= 1;
    }
  }
});

export const { increment, decrement } =
  counterSlice.actions;
export default counterSlice.reducer;
```

7.1.2 Step 2: Create the Store

```
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

```
export default store;
```

7.2 File Structure

```
/app
  /store
    store.js      (Create store here)
  /slices
    counterSlice.js (Reducers & actions)
    userSlice.js
  App.jsx
```

Best Practice: Organize slices in separate files!

Chapter 8

Providing the Store

8.1 Making Store Available

Your components need access to the store. Use a Provider:

```
import { Provider } from 'react-redux';
import store from './store/store';
import App from './App';

export function Root() {
  return (
    <Provider store={store}>
      <App />
    </Provider>
  );
}
```

8.2 In main.jsx or index.js

```
import React from 'react';
import ReactDOM from 'react-dom';
import { Provider } from 'react-redux';
import store from './store/store';
import App from './App';

ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

8.3 Why Provider?

The Provider:

- Makes store accessible to all components
- Uses React Context internally
- Wraps your entire app
- Must wrap before using Redux

Important: Provider is your connection to Redux!

Chapter 9

Using Redux Data in React Components

9.1 Reading State from Redux

Use the `useSelector` hook:

```
import { useSelector } from 'react-redux';

function Counter() {
  const count = useSelector(state => state.counter.value);

  return <div>Count: {count}</div>;
}
```

9.2 Understanding useSelector

```
const value = useSelector(state => state.path.to.data);
```

The function inside `useSelector`:

- Receives the entire state
- Returns the part you need
- Runs automatically when state changes
- Re-renders component if value changes

9.3 Different Selectors

```
// Get entire counter object
const counter = useSelector(state => state.counter);

// Get just the value
const count = useSelector(state => state.counter.value);
```

```
// Get user name
const name = useSelector(state => state.user.name);

// Get array
const items = useSelector(state => state.items);
```

9.4 Multiple Values

```
function Dashboard() {
  const count = useSelector(state => state.counter.value);
  const userName = useSelector(state => state.user.name);
  const items = useSelector(state => state.items);

  return (
    <div>
      <p>Hello {userName}, count: {count}</p>
      <p>Items: {items.length}</p>
    </div>
  );
}
```

Tip: Extract only the state you need for performance!

Chapter 10

Dispatching Actions From Inside Components

10.1 Sending Actions

Use the `useDispatch` hook:

```
import { useDispatch } from 'react-redux';

function Counter() {
  const dispatch = useDispatch();

  return (
    <button onClick={() => dispatch(increment())}>
      Increment
    </button>
  );
}
```

10.2 How It Works

1. Get `dispatch` from hook
2. Call `dispatch` with an action
3. Reducer handles the action
4. State updates
5. Component re-renders

10.3 Complete Example

```
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement } from '../slices/counterSlice';
```

```
function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch(increment())}>
        +
      </button>
      <button onClick={() => dispatch(decrement())}>
        -
      </button>
    </div>
  );
}

export default Counter;
```

10.4 Multiple Dispatches

```
function User() {
  const dispatch = useDispatch();

  const handleReset = () => {
    dispatch(clearCounter());
    dispatch(clearUser());
    dispatch(clearItems());
  };

  return <button onClick={handleReset}>Reset All</button>;
}
```

Pattern: useSelector to read, useDispatch to write!

Chapter 11

Redux with Class-based Components

11.1 Connecting Class Components

Class components can't use hooks. Use `connect()` instead:

```
import { connect } from 'react-redux';
import { increment } from '../slices/counterSlice';

class Counter extends React.Component {
  render() {
    return (
      <div>
        <p>Count: {this.props.count}</p>
        <button onClick={this.props.increment}>
          Increment
        </button>
      </div>
    );
  }
}

const mapStateToProps = (state) => ({
  count: state.counter.value
});

const mapDispatchToProps = {
  increment
};

export default connect(
  mapStateToProps,
  mapDispatchToProps
)(Counter);
```

11.2 mapStateToProps

Maps Redux state to component props:

```
const mapStateToProps = (state) => ({
  count: state.counter.value,
  userName: state.user.name
});
```

This makes:

```
this.props.count      // From state
this.props.userName    // From state
```

11.3 mapDispatchToProps

Maps actions to component props:

```
const mapDispatchToProps = {
  increment,
  decrement
};
```

```
// Now available as:
this.props.increment()
this.props.decrement()
```

Note: Class components are older. New code uses functional components with hooks!

Chapter 12

Attaching Payloads to Actions

12.1 What is a Payload?

A payload is extra data sent with an action.

```
// Action without payload
{ type: 'INCREMENT' }

// Action with payload
{ type: 'SET_COUNT', payload: 10 }

// Action with complex payload
{
  type: 'ADD_USER',
  payload: {
    name: 'John',
    age: 25,
    email: 'john@example.com'
  }
}
```

12.2 Creating Actions with Payloads

```
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    setCount: (state, action) => {
      state.value = action.payload;
    },
    addAmount: (state, action) => {
      state.value += action.payload;
    }
  }
});
```

```
export const { setCount, addAmount } = counterSlice.actions;
```

12.3 Using Payloads in Components

```
import { useDispatch } from 'react-redux';
import { setCount, addAmount } from '../slices/counterSlice';

function Counter() {
  const dispatch = useDispatch();

  return (
    <div>
      <button onClick={() => dispatch(setCount(100))}>
        Set to 100
      </button>
      <button onClick={() => dispatch(addAmount(5))}>
        Add 5
      </button>
    </div>
  );
}
```

12.4 Complex Payloads

```
const userSlice = createSlice({
  name: 'user',
  initialState: { users: [] },
  reducers: {
    addUser: (state, action) => {
      state.users.push(action.payload);
    }
  }
});

// In component:
dispatch(addUser({
  name: 'Jane',
  age: 28,
  email: 'jane@example.com'
})));
```

Tip: Pass data via payload, not global variables!

Chapter 13

Working with Multiple State Properties

13.1 Single Slice, Multiple Properties

```
const counterSlice = createSlice({
  name: 'counter',
  initialState: {
    value: 0,
    history: [],
    step: 1
  },
  reducers: {
    increment: (state) => {
      state.value += state.step;
      state.history.push(state.value);
    },
    setStep: (state, action) => {
      state.step = action.payload;
    }
  }
});
```

13.2 Selecting Multiple Properties

```
function Counter() {
  const value = useSelector(state => state.counter.value);
  const step = useSelector(state => state.counter.step);
  const history = useSelector(state => state.counter.history);

  return (
    <div>
      <p>Count: {value}</p>
      <p>Step: {step}</p>
      <p>History: {history.join(', ')}</p>
    </div>
  );
}
```

```
    </div>
  );
}
```

13.3 Select Entire Object

```
const counterData = useSelector(state => state.counter);

// Access with:
counterData.value
counterData.step
counterData.history
```

13.4 Multiple Slices

Store with many slices:

```
const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer,
    items: itemsReducer,
    ui: uiReducer
  }
});
```

13.5 Accessing Multiple Slices

```
function Dashboard() {
  const count = useSelector(state => state.counter.value);
  const userName = useSelector(state => state.user.name);
  const items = useSelector(state => state.items.list);
  const isDarkMode = useSelector(state => state.ui.darkMode);

  return (
    <div className={isDarkMode ? 'dark' : 'light'}>
      <p>Hello {userName}, count: {count}</p>
      <p>Items: {items.length}</p>
    </div>
  );
}
```

Organization: Organize properties logically into slices!

Chapter 14

How To Work With Redux State Correctly

14.1 The Immutability Rule

NEVER modify state directly!

```
// BAD: Direct mutation
state.counter = state.counter + 1;
state.user.name = 'John';

// GOOD: Return new state
return { ...state, counter: state.counter + 1 };
return { ...state, user: { ...state.user, name: 'John' } };
```

14.2 Redux Toolkit Simplifies This

Redux Toolkit uses Immer, which lets you write mutating code that's actually immutable:

```
// This looks like mutation but is actually immutable!
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => {
      state.value += 1; // Looks like mutation
    }
  }
});

// Immer transforms it to immutable code behind the scenes
```

14.3 Common Patterns

14.3.1 Adding to Array

```
// Redux Toolkit way (Immer handles it)
addItem: (state, action) => {
  state.items.push(action.payload);
}

// Pure JS way
addItem: (state, action) => {
  return {
    ...state,
    items: [...state.items, action.payload]
  };
}
```

14.3.2 Removing from Array

```
// Redux Toolkit
removeItem: (state, action) => {
  state.items = state.items.filter(
    item => item.id !== action.payload
  );
}

// Pure JS
removeItem: (state, action) => {
  return {
    ...state,
    items: state.items.filter(
      item => item.id !== action.payload
    )
  };
}
```

14.3.3 Updating Nested Objects

```
// Redux Toolkit
updateUser: (state, action) => {
  state.user.name = action.payload.name;
}

// Pure JS
updateUser: (state, action) => {
  return {
    ...state,
    user: { ...state.user, name: action.payload.name }
  };
}
```



```
}
```

Critical: Immutability prevents bugs and enables debugging!

Chapter 15

Adding State Slices

15.1 What is a Slice?

A slice is a piece of your Redux state. It contains:

- Initial state
- Reducers (functions that update state)
- Actions (automatically created)

15.2 Creating a New Slice

```
import { createSlice } from '@reduxjs/toolkit';
```

```
const userSlice = createSlice({
  name: 'user',
  initialState: {
    id: null,
    name: '',
    email: '',
    isLoggedIn: false
  },
  reducers: {
    login: (state, action) => {
      state.id = action.payload.id;
      state.name = action.payload.name;
      state.email = action.payload.email;
      state.isLoggedIn = true;
    },
    logout: (state) => {
      state.id = null;
      state.name = '';
      state.email = '';
      state.isLoggedIn = false;
    },
  },
});
```

```
    updateName: (state, action) => {
      state.name = action.payload;
    }
  }
});

export const { login, logout, updateName } = userSlice.actions;
export default userSlice.reducer;
```

15.3 Adding to Store

```
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';
import userReducer from './userSlice';

const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer
  }
});

export default store;
```

15.4 Using the Slice

```
import { useSelector, useDispatch } from 'react-redux';
import { login, logout } from '../slices/userSlice';

function User() {
  const user = useSelector(state => state.user);
  const dispatch = useDispatch();

  const handleLogin = () => {
    dispatch(login({
      id: 1,
      name: 'John',
      email: 'john@example.com'
    }));
  };

  const handleLogout = () => {
    dispatch(logout());
  };

  return (
    <div>
```

```
{user.isLoggedIn ? (  
  <>  
    <p>Welcome {user.name}</p>  
    <button onClick={handleLogout}>Logout</button>  
  </>  
) : (  
  <button onClick={handleLogin}>Login</button>  
)}  
</div>  
);  
}
```

Best Practice: Create one slice per feature!

Chapter 16

Connecting Redux Toolkit State

16.1 Step 1: Create Slice

```
// slices/itemSlice.js
import { createSlice } from '@reduxjs/toolkit';

const itemSlice = createSlice({
  name: 'items',
  initialState: {
    list: [],
    loading: false,
    error: null
  },
  reducers: {
    setLoading: (state) => {
      state.loading = true;
    },
    setItems: (state, action) => {
      state.list = action.payload;
      state.loading = false;
    },
    setError: (state, action) => {
      state.error = action.payload;
      state.loading = false;
    }
  }
});

export const { setLoading, setItems, setError } = itemSlice.actions;
export default itemSlice.reducer;
```

16.2 Step 2: Add to Store

```
// store/store.js
import { configureStore } from '@reduxjs/toolkit';
```

```
import itemReducer from '../slices/itemSlice';

const store = configureStore({
  reducer: {
    items: itemReducer
  }
});

export default store;
```

16.3 Step 3: Use in Component

```
// components/Items.jsx
import { useSelector, useDispatch } from 'react-redux';
import { setLoading, setItems, setError } from '../slices/itemSlice';
import { useEffect } from 'react';

function Items() {
  const dispatch = useDispatch();
  const { list, loading, error } = useSelector(state => state.items);

  useEffect(() => {
    const fetchItems = async () => {
      dispatch(setLoading());
      try {
        const response = await fetch('/api/items');
        const data = await response.json();
        dispatch(setItems(data));
      } catch (err) {
        dispatch(setError(err.message));
      }
    };

    fetchItems();
  }, [dispatch]);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <ul>
      {list.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}
```

```
export default Items;
```

Flow: Slice → Store → useSelector/useDispatch → Component

Chapter 17

Working with Multiple Slices

17.1 Many Slices Strategy

Break your state into logical slices:

```
/slices
  counterSlice.js
  userSlice.js
  itemsSlice.js
  uiSlice.js
  authSlice.js
```

17.2 Each Slice Example

counterSlice.js:

```
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: { /* ... */ }
});
```

userSlice.js:

```
const userSlice = createSlice({
  name: 'user',
  initialState: { name: '', email: '' },
  reducers: { /* ... */ }
});
```

itemsSlice.js:

```
const itemsSlice = createSlice({
  name: 'items',
  initialState: { list: [] },
  reducers: { /* ... */ }
});
```


17.3 Combining in Store

```
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';
import userReducer from './userSlice';
import itemsReducer from './itemsSlice';

const store = configureStore({
  reducer: {
    counter: counterReducer,
    user: userReducer,
    items: itemsReducer
  }
});

export default store;
```

17.4 State Shape

Your Redux state looks like:

```
{
  counter: { value: 0 },
  user: { name: '', email: '' },
  items: { list: [] }
}
```

17.5 Using Multiple Slices

```
function Dashboard() {
  const count = useSelector(state => state.counter.value);
  const user = useSelector(state => state.user);
  const items = useSelector(state => state.items.list);

  return (
    <div>
      <p>User: {user.name}</p>
      <p>Count: {count}</p>
      <p>Items: {items.length}</p>
    </div>
  );
}
```

Organization: Each slice = one feature!

Chapter 18

Reading & Dispatching From A New Slice

18.1 Complete Example: Todo Slice

18.1.1 Create Todo Slice

```
// slices/todoSlice.js
import { createSlice } from '@reduxjs/toolkit';

const todoSlice = createSlice({
  name: 'todos',
  initialState: {
    todos: [],
    filter: 'all' // 'all', 'active', 'completed'
  },
  reducers: {
    addTodo: (state, action) => {
      state.todos.push({
        id: Date.now(),
        text: action.payload,
        completed: false
      });
    },
    removeTodo: (state, action) => {
      state.todos = state.todos.filter(
        todo => todo.id !== action.payload
      );
    },
    toggleTodo: (state, action) => {
      const todo = state.todos.find(t => t.id === action.payload);
      if (todo) todo.completed = !todo.completed;
    },
    setFilter: (state, action) => {
      state.filter = action.payload;
    }
  }
});
```

```
    }  
  });  
  
export const { addTodo, removeTodo, toggleTodo, setFilter } =  
  todoSlice.actions;  
export default todoSlice.reducer;
```

18.1.2 Add to Store

```
import { configureStore } from '@reduxjs/toolkit';  
import todoReducer from './todoSlice';
```

```
const store = configureStore({  
  reducer: {  
    todos: todoReducer  
  }  
});
```

```
export default store;
```

18.1.3 Use in Component

```
import { useSelector, useDispatch } from 'react-redux';  
import { addTodo, removeTodo, toggleTodo }  
  from '../slices/todoSlice';  
  
function TodoApp() {  
  const dispatch = useDispatch();  
  const todos = useSelector(state => state.todos.todos);  
  const filter = useSelector(state => state.todos.filter);  
  
  const handleAddTodo = (text) => {  
    dispatch(addTodo(text));  
  };  
  
  const handleRemove = (id) => {  
    dispatch(removeTodo(id));  
  };  
  
  const handleToggle = (id) => {  
    dispatch(toggleTodo(id));  
  };  
  
  const filteredTodos = todos.filter(todo => {  
    if (filter === 'completed') return todo.completed;  
    if (filter === 'active') return !todo.completed;  
    return true;  
  });  
};
```

```
return (
  <div>
    <input
      onPress={(e) => {
        if (e.key === 'Enter') {
          handleAddTodo(e.target.value);
          e.target.value = '';
        }
      }}
    />

    {filteredTodos.map(todo => (
      <div key={todo.id}>
        <input
          type="checkbox"
          checked={todo.completed}
          onChange={() => handleToggle(todo.id)}
        />
        <span>{todo.text}</span>
        <button onClick={() => handleRemove(todo.id)}>
          Delete
        </button>
      </div>
    ))}
  </div>
);
}

export default TodoApp;
```

Complete Pattern: Slice → Store → useSelector/useDispatch → Component

Quick Summary

Redux Fundamentals

1. **State** - Central data storage
2. **Actions** - Describe what happened
3. **Reducers** - Update state
4. **Store** - Combines everything
5. **Dispatch** - Send actions

Redux Toolkit Essentials

- `createSlice()` - Create actions + reducers
- `configureStore()` - Create store
- `useSelector` - Read state
- `useDispatch` - Send actions
- `Provider` - Make store available

Best Practices

1. Always use Redux Toolkit (not plain Redux)
2. Keep state immutable
3. Organize into logical slices
4. Use meaningful action names
5. Send payloads with actions
6. Validate data before storing
7. Keep reducers pure
8. Organize files by feature

File Structure

```
/src
  /store
    store.js
  /slices
    counterSlice.js
    userSlice.js
    itemsSlice.js
  /components
    Counter.jsx
    User.jsx
  App.jsx
```

Golden Rules

1. **Single Store** - One store for entire app
2. **Read-Only State** - Only change via reducers
3. **Pure Reducers** - Same input = same output
4. **Use Redux Toolkit** - Not plain Redux
5. **Think Before Using** - Use Context for simple apps

When to Use Redux

Use Redux when:

- App has lots of state
- Many components need same data
- State changes are complex
- You need time-travel debugging
- Working in a large team

Skip Redux if:

- Small app (few components)
- Simple state (theme, user)
- Just learning React

Redux makes managing complex state predictable and debuggable!